

Chapter 7 — Physical limits

Tokens were the unit of meaning. Here they become the unit of price.

This is a **governing-constraint analysis**. You already do this with load paths and limit states: find the binding constraint, then reason about what relaxes it.

What this chapter does not tell you

This chapter is about **cost** and **latency**. It says nothing about **accuracy**.

- There is an equation for what a long context costs. You are about to derive it, and it closes.
- There is no equation for what a long context does to accuracy. There are empirical curves and mechanistic arguments, and they are Chapter 5.
- Conflating the two is the easiest mistake to make with this material, precisely because the cost result is clean and the accuracy result is not.

A long context is expensive and slow. Whether it is also **wrong** is a different question with a different kind of answer.

Where this comes from, and who benefits from it

The construction that follows is from a blackboard lecture, published as a podcast transcript.

- It is **expert testimony**, not a peer-reviewed result. The footer says so on every slide that uses it.
- The speaker is CEO of a chip company. The interviewer discloses being an angel investor in that company.
- The lecture's central conclusion — that **memory bandwidth is the binding constraint** — is commercially convenient for both of them.

The analysis still appears sound, and you are about to check the algebra yourself.

Declaring an interest is not the same as rejecting the work. Doing neither is what gets you in trouble, and it is the habit Chapter 18 asks of you in your own writing.

Symbols

All quantities positive definite — no sign convention is required. SI throughout.

Symbol	Meaning	Unit
B	batch size, tokens per forward pass	—
N_{act}	active parameters per token	—
N_{tot}	total parameters	—
F	arithmetic throughput	FLOP s^{-1}
M	throughput in multiply-accumulates, $1 \text{ MAC} = 2 \text{ FLOP}$	MAC s^{-1}
b_w	bytes per stored parameter	byte

The roofline bound

$$T \geq \max(T_{\text{comp}}, T_{\text{mem}})$$

$$T_{\text{comp}} = \frac{2 B N_{\text{act}}}{F} = \frac{B N_{\text{act}}}{M}$$

$$T_{\text{mem}} = \frac{\overbrace{N_{\text{tot}} b_w}^{\text{weight fetch}} + \overbrace{B L b_{\text{kv}}}^{\text{KV fetch}}}{\beta}$$

$$c \equiv \frac{T}{B}$$

$$\frac{T_{\text{mem}}}{B} = \underbrace{\frac{N_{\text{tot}} b_w}{\beta B}}_{\propto 1/B} + \underbrace{\frac{L b_{\text{kv}}}{\beta}}_{\text{constant in } B}$$

Dimensions. $2BN_{\text{act}}$ is a FLOP count, so T_{comp} is $\text{FLOP}/(\text{FLOP s}^{-1}) = \text{s}$. $N_{\text{tot}}b_w + BLb_{\text{kv}}$ is bytes, so T_{mem} is $\text{byte}/(\text{byte s}^{-1}) = \text{s}$. Cost c is s token^{-1} .

What the hyperbola says

The weight-fetch term carries $1/B$. At $B = 1$ every parameter in the model is read from memory to produce one token.

As B grows that term amortises and cost flattens onto the KV floor $L b_{\text{kv}}/\beta$, which B does not help — it is linear in context length and paid per token regardless.

Consequence. Long context is bounded by memory bandwidth and capacity, not by compute. That is a different claim from anything about accuracy, which has no equation and is Chapter 5's material.

Critical batch size

$$T_{\text{comp}} = T_{\text{mem}}^{(\text{weight})}$$

$$\frac{2 B^* N_{\text{act}}}{F} = \frac{N_{\text{tot}} b_w}{\beta}$$

$$B^* = \underbrace{\frac{F b_w}{2 \beta}}_{\text{hardware ratio}} \times \underbrace{\frac{N_{\text{tot}}}{N_{\text{act}}}}_{\text{sparsity}}$$

The source reports the hardware ratio as ≈ 300 , dimensionless and stable across several GPU generations. Critical batch size is then $\approx 300 \times \text{sparsity}$.

The denominator is the whole argument

T_{comp} was written twice above, and the two forms are not interchangeable:

$$T_{\text{comp}} = \frac{B N_{\text{act}}}{M} \quad M \text{ in MAC s}^{-1}$$

$$T_{\text{comp}} = \frac{2 B N_{\text{act}}}{F} \quad F \text{ in FLOP s}^{-1}$$

Substituting a spec-sheet FLOP s^{-1} into the MAC-denominated form makes the answer **wrong by exactly a factor of two**. An outside reader caught this in the transcript's comment thread.

A unit-definition error, in an expert lecture, found by peer review, producing a clean factor of two. Chapter 4 uses this as its anchor.

Two numbers you will see quoted

From the compute term already on the board, a forward pass costs about **two FLOP per parameter per token** — one multiply and one add per weight.

$$\text{forward} \approx 2ND \quad [\text{FLOP}]$$

N — parameters, D — tokens processed. Both dimensionless counts.

- That is not a new result. It is the numerator of T_{comp} , read at the scale of a whole training run instead of one batch.
- Training is conventionally accounted at **three times** the forward cost — forward, plus a backward pass computing gradients with respect to both activations and weights — giving $6ND$.

TODO(cite) — $2ND$ follows from what is already on the board. The factor of three does not, and is standard accounting rather than something derived here. Search: `6ND training FLOPs derivation` · `Chinchilla compute optimal 6ND` · `Kaplan scaling laws compute budget` .

What has to fit on the device

$$M_{\text{device}} = \frac{N_{\text{tot}} b_w + B L b_{\text{kv}}}{E_p \times P_p} \quad [\text{byte}]$$

E_p — expert parallelism, P_p — pipeline parallelism. Both dimensionless counts.

- The same two terms as before — weights, and KV — now divided across devices.
- **Note the units.** The source states this as *parameters plus KV bytes*. Parameters are a count; only $N_{\text{tot}} b_w$ is a quantity of bytes. The formula is right once you read it that way, and this is exactly the denominator problem from three slides ago wearing a different hat.

Pipelining does not help the KV cache

- Splitting the model across pipeline stages reduces **weight** storage per rack. That part works.
- It does **not** reduce KV storage. The source states that the pipeline-stage factor cancels.

TODO(verify) — the source states the *result*, not the mechanism. A full pipeline holds one micro-batch per stage, so the batch in flight would scale with the number of stages and cancel the division. That is a reading, not a quotation. Check it against the transcript before teaching it as the reason.

Long context is a memory problem, not a compute problem

- **Bandwidth.** The KV cache is re-read for every token generated. That term is linear in L .
 - **Capacity.** The KV cache has to fit. That term is linear in L too.
 - Compute is not what binds you at long context. Memory is, at both ends.
 - Sparse attention improves the scaling — the source notes it does so at a cost in quality.
-

Thread — cost. This closes the first half. Chapter 10 spends it on resident context, Chapter 11 on the orchestration premium, Chapter 13 on what you are actually billed.

Energy: what can and cannot be said

What can be said

- Training is a one-off cost. Inference is a recurring cost per query.
- They are different quantities with different units, and they are routinely added together.
- Both are real, and both are large.

What cannot be said here

- Any per-query figure at all.
- Published numbers are contested and routinely conflate one-off training cost with per-query inference.

This slide has no number on it, deliberately. A number you cannot defend is worse than no number, in front of a room that will ask where it came from.

Three alternative substrates. They are not equivalent.

Approach	What has actually been shown	Standing
Invertible architectures	Activations reconstructed from later ones; storage independent of depth at nearly identical accuracy	Published result — but a training-time technique
Neuromorphic hardware	Large energy gains on specific benchmark tasks	Lab demonstration; not commercially deployed
Organoid computing	Tissue that fires, responds to stimulation, and oscillates	No learning system reported — the authors' own words

These three get roughly equal airtime in most talks about the future of compute. They have not earned it. The differences in the right-hand column are the content of this section, and stating them is the point.

Trading compute for memory

Reversible architectures reconstruct each layer's activations from the next layer's, so they need not be stored. Activation storage becomes independent of depth, at nearly identical accuracy.

- You spend FLOPs to buy back bytes. Against a memory-bound system, that is the right direction to trade.
- **But this is a training-time result.** It addresses storing activations during backpropagation, not the inference-time KV cache this chapter has been pricing.
- Keep the parallel; do not let it slide into a claim about inference.

Neuromorphic hardware

Real engineering, real gains, and a gap between the lab and your problem.

- Reported: image classification at a fraction of the energy and several times faster; a neuromorphic language model matching a comparable GPU-based one's accuracy at half the energy.
- **Not commercially deployed.** The source describes the leading chips as demonstration and experimentation tools.
- The gains are benchmark-specific and are not shown to transfer to frontier-scale systems. The source states these chips cannot simply be dropped into today's LLM stacks.

An earlier draft of this course called this "deployed, engineering-grade." Checking the source corrected it. That correction is worth more to you than the claim was.

Organoid and wetware computing

- Demonstrated: spontaneous electrophysiological activity, response to stimulation, myelinated axons, oscillatory behaviour, multiple cell types at realistic densities.
- Proposed: training via biofeedback, interfacing with sensors, decoding responses into computational output.

From the foundational paper itself:

To the best of our knowledge, however, no relevant approach using brain organoids as learning systems has been reported.

- Organoids are avascular, with necrosis beyond roughly 300 μm . They have no predictable anatomy or defined topography.
- What exists is **tissue characterisation, not computation**. "Early stage, modest results" overstates it.

Where this leaves the cost thread

- Time is bounded below by the larger of compute time and memory time, and for the workloads you will run, memory wins.
- Cost per token is a hyperbola in batch size. Batch one is the expensive corner, and it is the corner you sit in.
- Long context is bounded by memory bandwidth and capacity, not by arithmetic.
- A denominator defined as multiply-accumulates and read as FLOPs is wrong by exactly two — and that was caught by a reader, not by the expert.

None of this says anything about whether the answer is **correct**.

That is Chapter 5, and it has no equation.